

FOVL

Deeksha Varshney

Word Vectors

Content is taken from various sources

Representing words as discrete symbols

In traditional NLP, we regard words as discrete symbols:

hotel, conference, motel – a localist representation

Means one 1, the rest 0s

Such symbols for words can be represented by one-hot vectors:

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

Vector dimension = number of words in vocabulary (e.g., 500,000+)

Problem with words as discrete symbols

Example: in web search, if a user searches for "Seattle motel", we would like to match documents containing "Seattle hotel"

But:

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]
hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

These two vectors are orthogonal

There is no natural notion of **similarity** for one-hot vectors!

- $\text{hotel} \perp \text{motel}$ (dot product = 0)

All words equally distant, No semantic structure, Inefficient and sparse

Solution:

- Could try to rely on WordNet's list of synonyms to get similarity?
  - But it is well-known to fail badly: Synonym Lists Are Incomplete, Similarity Is Not Binary, Context Changes Meaning, etc.
- **Instead: learn to encode similarity in the vectors themselves**

Representing words by their context



- Distributional semantics: **A word's meaning is given by the words that frequently appear close-by**
  - *"You shall know a word by the company it keeps"* (J. R. Firth 1957: 11)
  - One of the most successful ideas of modern statistical NLP!
- When a word *w* appears in a text, its **context** is the set of words that appear nearby (within a fixed-size window).
- We use the many contexts of *w* to build up a representation of *w*

…government debt problems turning into	***banking***	*crises as happened in 2009…*
…saying that Europe needs unified	***banking***	*regulation to replace the hodgepodge…*
…India has just given its	***banking***	*system a shot in the arm…*

These context words will represent ***banking***

Word vectors

We will build a dense vector for each word, chosen so that it is similar to vectors of words that appear in similar contexts, measuring similarity as the vector dot (scalar) product

$$\textit{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix} \qquad \textit{monetary} = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

Note: word vectors are also called (word) embeddings or (neural) word representations
They are a distributed representation

Word meaning as a neural word vector – visualization

expect =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$



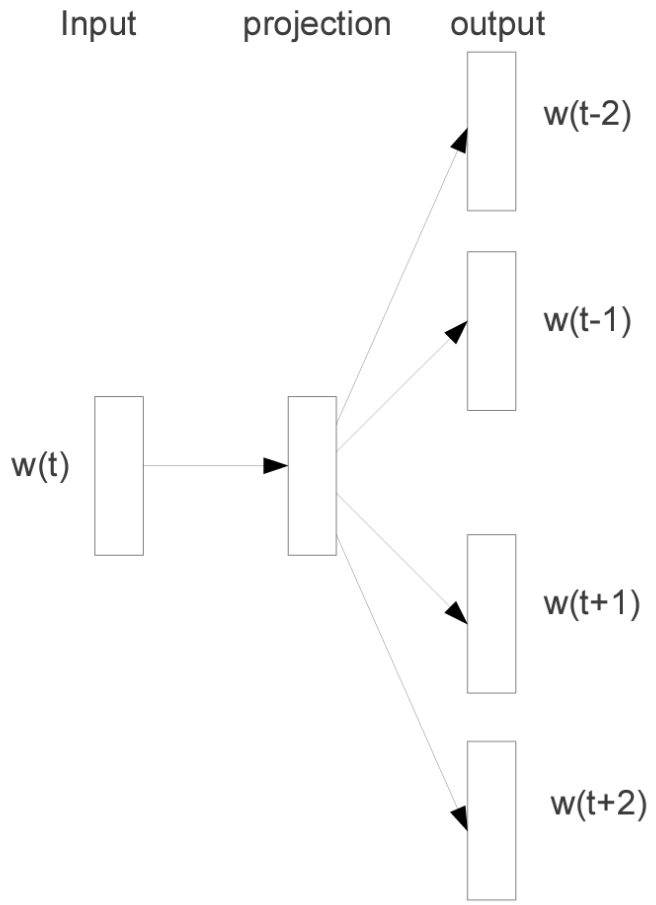
3. Word2vec: Overview

Word2vec is a framework for learning word vectors

(Mikolov et al. 2013)*

Idea:

- We have a large corpus ("body") of text: a long list of words
- Every word in a fixed vocabulary is represented by a vector
- Go through each position *t* in the text, which has a center word *c* and context ("outside") words *o*
- Use the similarity of the word vectors for *c* and *o* to calculate the probability of *o* given *c* (or vice versa)
- Keep adjusting the word vectors to maximize this probability

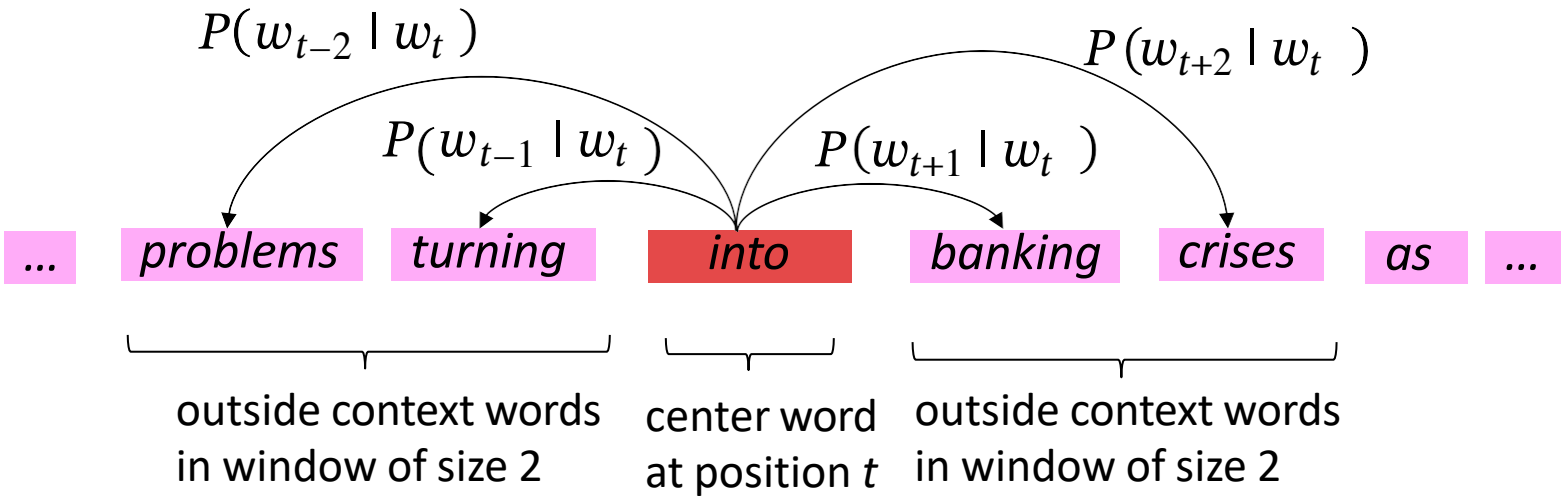


Skip-gram model
(Mikolov et al. 2013)

*https://arxiv.org/pdf/1301.3781

Word2Vec Overview

Example windows and process for computing $P\left(w_{t+\mathrm{j}} \mid w_t\right)$

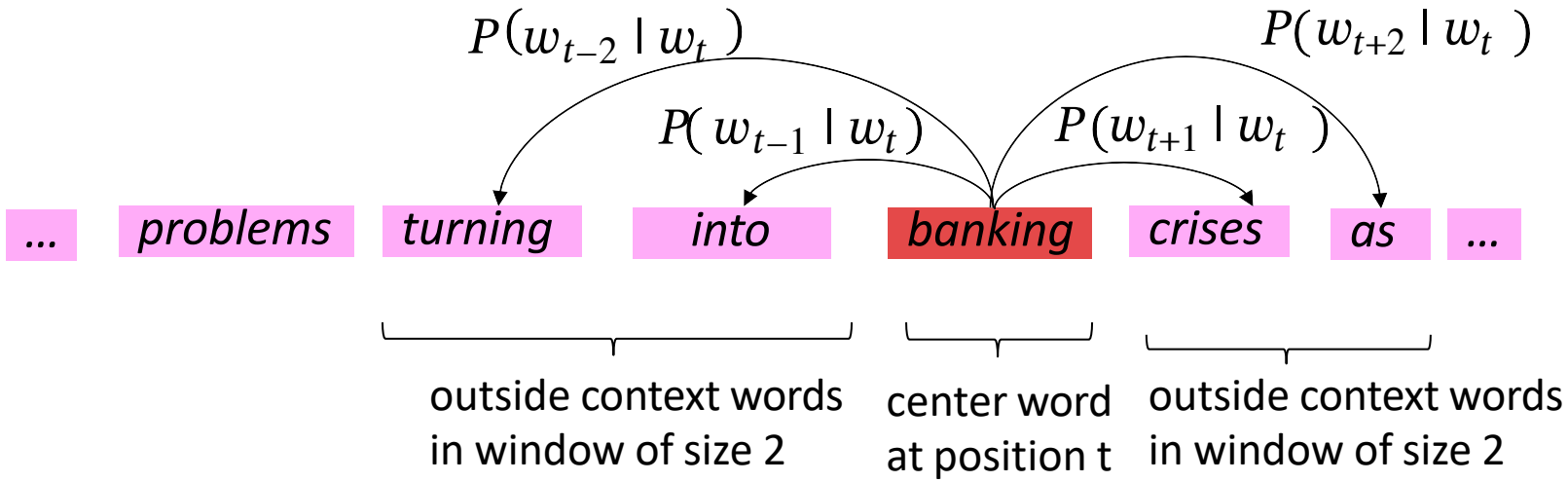


We slide a **context window** over the text.

Example window size = 2
So for each **center word**, we look at 2 words **before and after it**.

Word2Vec Overview

Example windows and process for computing $P\left(w_{t+\mathrm{j}} \mid w_t\right)$



Word2vec: objective function

For each position $t = 1, \ldots, T$, predict context words within a window of fixed size m, given center word w_t. Data likelihood:

$$\text{Likelihood} = L(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \le j \le m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta)$$

θ is all variables to be optimized

According to our current word vectors θ, how likely is this context word given the center word?"

sometimes called a *cost* or *loss* function

The objective function $J(\theta)$ is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-m \le j \le m \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

Minimizing objective function ⟺ Maximizing predictive accuracy

Word2vec: objective function

- We want to minimize the objective function:

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{\substack{-m\le j\le m \\ j\neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

- **Question:** How to calculate $P(w_{t+j} \mid w_t; \theta)$?
- **Answer:** We will *use two* vectors per word *w*:
  - v_w when *w* is a center word
  - u_w when *w* is a context word

"These are not two meanings of the word.
They are **two roles** the word can play."

- Then for a center word *c* and a context word *o*:

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w\in V}\exp(u_w^T v_c)}$$

We want an **objective function** that:
- Is easy to optimize
- Works well numerically

So we do three transformations:

Step 1: Log
- Products → sums
- Avoids numerical underflow

Step 2: Negative sign
- Optimization algorithms **minimize**
- So we minimize **negative log likelihood**

Step 3: Average
Divide by number of words T

Word2vec: prediction function

② Exponentiation makes anything positive

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

① Dot product compares similarity of *o* and *c*.
$u^T v = u.v = \sum_{i=1}^{n} u_i v_i$
Larger dot product = larger probability

③ Normalize over entire vocabulary to give probability distribution

- This is an example of the **softmax function** $\mathbb{R}^n \to (0,1)^n$ ← Open region

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^{n} \exp(x_j)} = p_i$$

- The softmax function maps arbitrary values x_i to a probability distribution p_i
  - "max" because amplifies probability of largest x_i
  - "soft" because still assigns some probability to smaller x_i
  - Frequently used in Deep Learning

But sort of a weird name because it returns a distribution!

Step ① Dot Product = Similarity

$$u_o^\top v_c$$

- Measures **how compatible** the center word and context word are
- Larger dot product ⇒ more similar
- Similar words ⇒ higher probability

Intuition:

- `banking` ↔ `crises` → large dot product
- `banking` ↔ `skillet` → small dot product

Step ② Exponentiation

$$\exp(u_o^\top v_c)$$

Why do this?

- Dot products can be negative
- Probabilities must be **positive**
- Exponentiation guarantees positivity

Also:

- Large similarities become **much larger**
- Small similarities shrink rapidly

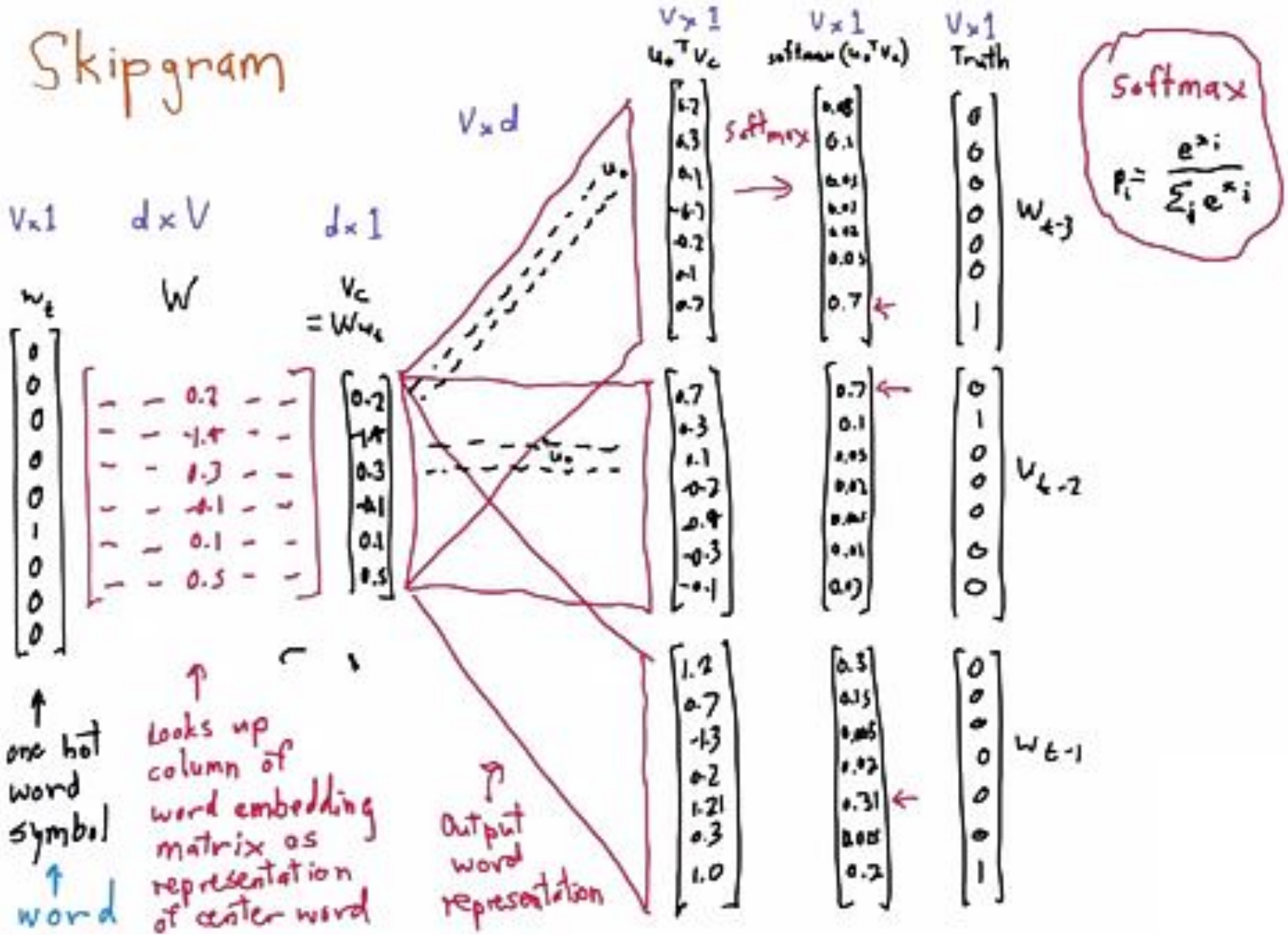
Step ③ Normalize (Softmax)

We divide by the sum over **all words in the vocabulary**.

$$\sum_{w \in V} \exp(u_w^\top v_c)$$

This ensures:

- All probabilities sum to 1
- We get a valid probability distribution


Skipgram
V×1
d×V
d×1
V×d
V×1
V×1
V×1
w_t
W
$v_c = Wu_c$
$u_o^T v_c$
softmax
softmax($u_o^T v_c$)
Truth
Softmax
$p_i = \frac{e^{x_i}}{\sum_i e^{x_i}}$
W_{t-1}
W_{t-2}
W_{t-1}
one hot word symbol
word
Looks up column of word embedding matrix as representation of center word
Output word representation

- “In Skip-gram, the neural network takes a one-hot vector for the center word as input, which simply selects one column from the embedding matrix to produce the center word vector.
- This vector is then compared, using dot products, with every output word vector to produce a score for each word in the vocabulary.
- A softmax converts these scores into probabilities that represent how likely each word is to appear near the center word.
- The model is trained to assign high probability to the true context words and low probability to others, and through gradient descent, both the input (center) and output (context) word vectors are updated.
- Over time, words that appear in similar contexts learn similar vectors, which is how semantic meaning emerges.”

WORD2VEC-SKIPGRAM(Corpus, c, d, η, T)

1. Build vocabulary V from the corpus
2. Initialize input embeddings E randomly
3. Initialize output weights W randomly

4. Generate training pairs:
   For each sentence in Corpus:
      For each word w at position i:
         For each context position j within window c:
            If i ≠ j:
               Add pair (w, context_word)

5. For epoch = 1 to T:
      Shuffle training pairs

      For each pair (center, context):

         # Forward pass
         h = E[center]
         scores = h × W
         probabilities = Softmax(scores)

         # Compute loss
         loss = -log(probabilities[context])

         # Backpropagation
         error = probabilities
         error[context] -= 1

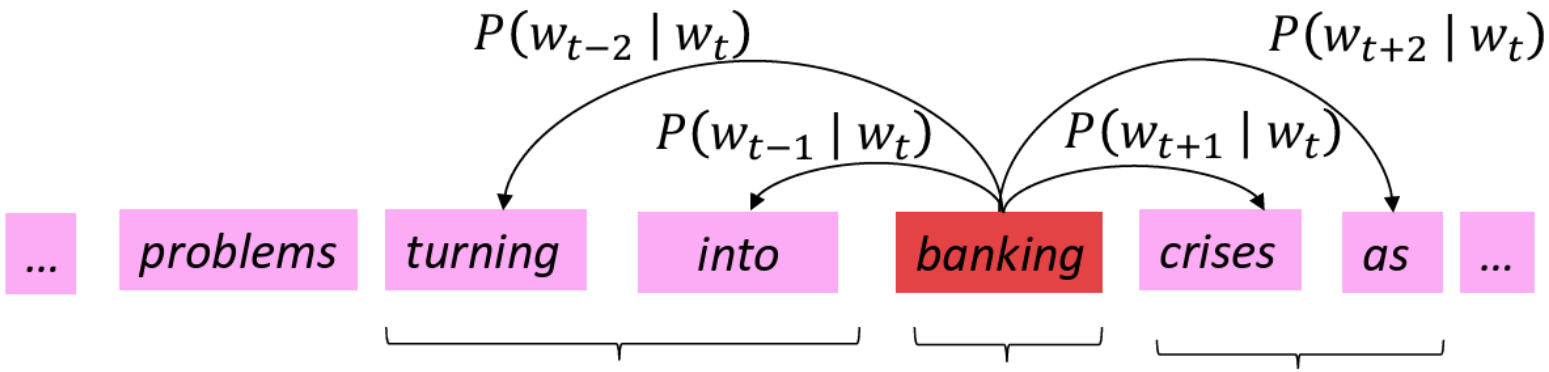
         gradient_W = h^T × error
         gradient_h = W × error

         # Update parameters
         W = W - η × gradient_W
         E[center] = E[center] - η × gradient_h

6. Return E

Review: Main idea of word2vec

- Start with random word vectors
- Iterate through each word position in the whole corpus
- Try to predict surrounding words using word vectors: $P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$



- **Learning:** Update vectors so they can predict actual surrounding words better
- Doing no more than this, this algorithm learns word vectors that capture well word similarity and meaningful directions in a word space!

Stochastic Gradient Descent

- **Problem**: $J(\theta)$ is a function of **all** windows in the corpus (potentially billions!)
  - So $\nabla_\theta J(\theta)$ is very expensive to compute
- You would wait a very long time before making a single update!

- **Very** bad idea for pretty much all neural nets!
- **Solution: Stochastic gradient descent (SGD)**
  - Repeatedly sample windows, and update after each one
- Algorithm:

Mini Batch Gradient Descent

```
while True:
    window = sample_window(corpus)
    theta_grad = evaluate_gradient(J,window,theta)
    theta = theta - alpha * theta_grad
```

Word2vec algorithm variants

Why two vectors? → Easier optimization. Average both at the end

Two model variants:

1. Skip-grams (SG)

   Predict context ("outside") words (position independent) given center word

2. Continuous Bag of Words (CBOW)

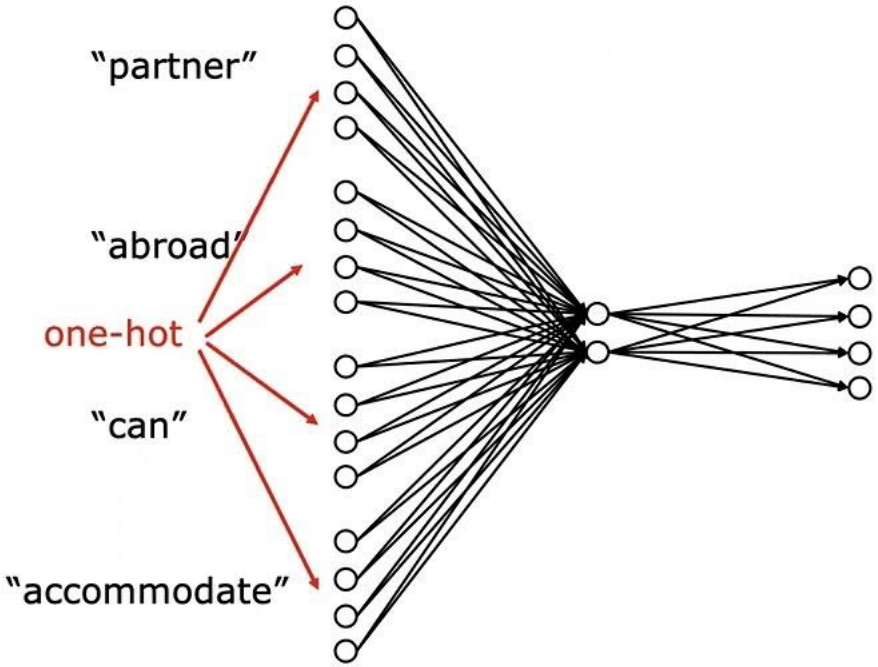
   Predict center word from (bag of) context words

Loss functions for training:

1. Naïve softmax (simple but expensive loss function, when many output classes)
2. More optimized variants like hierarchical softmax
3. Negative sampling

CBOW (Mikolov et al. 2013)

- Given context words, predict the target word



Model	What it predicts	What it learns
Skip-gram (Word2Vec)	Given a target word, predict its context words	Word vectors that capture co-occurrence patterns
CBOW (Word2Vec)	Given context words, predict the target word	Similar word vectors
GloVe	Learn from co-occurrence counts	Word vectors encoding statistical relationships

The skip-gram model with negative sampling

- The normalization term is computationally expensive (when many output classes):

- $$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

A big sum over words. The denominator sums over **all words** in the vocabulary.

This denominator is **huge** (maybe 100k+ words)!
So computing it for every update is **too slow**.

- Hence, in standard word2vec you implement the skip-gram model with **negative sampling**

- Main idea: train binary logistic regressions to differentiate a true pair (center word and a word in its context window) versus several "noise" pairs (the center word paired with a random word)

Skip-gram with negative sampling (Word2Vec)

Example

Sentence: "*The cat sat on the mat*"

For center word **"cat"**, positive context words might be:
→ ["the", "sat", "on"]

Negative samples (random):
→ ["apple", "car", "quickly"]

Training updates:

- Bring (“cat”, “sat”) **closer**
- Push (“cat”, “apple”) **farther apart**

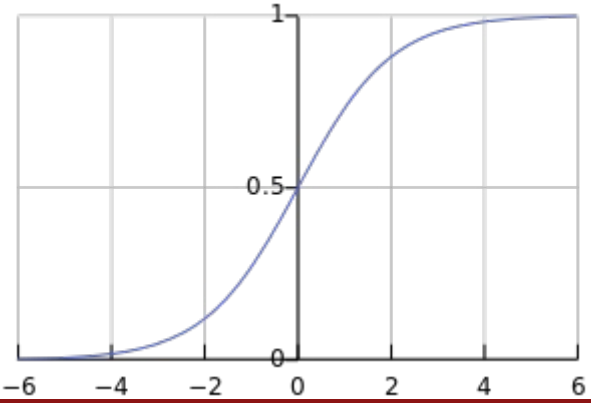
The skip-gram model with negative sampling

- Introduced in: “Distributed Representations of Words and Phrases and their Compositionality*” (Mikolov et al. 2013)
- Overall objective function (they maximize): $J(\theta) = \frac{1}{T}\sum_{t=1}^{T} J_t(\theta)$

$$J_t(\theta) = \log \sigma\left(u_o^T v_c\right) + \sum_{i=1}^{k} \mathbb{E}_{j\sim P(w)}\left[\log \sigma\left(-u_j^T v_c\right)\right]$$

sigmoid rather than softmax

- The logistic/sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$
- We maximize the probability of two words co-occurring in first log and minimize probability of noise words in second part



*http://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.pdf

$$J_t(\theta) = \log \sigma(u_o^T v_c) + \sum_{j=1}^{k} \mathbb{E}_{j \sim P(w)} \left[\log \sigma(-u_j^T v_c) \right]$$

Explanation of Each Term

The term $J_t(\theta)$ represents the objective for one center-context word pair in the training data. Here, θ denotes all model parameters, namely the word embedding vectors that the model learns.

The vector v_c is the **input (center word) embedding**. If the center word is "king", then v_c is the learned vector representation of "king".

The sigmoid function is defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

which maps any real number to a probability between 0 and 1.
The first term,

$$\log \sigma(u_o^T v_c),$$

encourages the model to assign a high probability to the observed (positive) center-context pair. If "king" and "queen" frequently co-occur, the model increases their dot product, making this term larger.

The second term,

$$\sum_{j=1}^{k} \mathbb{E}_{j\sim P(w)} \left[\log \sigma(-u_j^T v_c) \right],$$

handles **negative sampling**. Instead of comparing the center word against every word in the vocabulary (as softmax does), the model samples k random words that are unlikely to be correct context words.
Here:

- k = number of negative samples.
- u_j = output vector of the jth negative word.
- $P(w)$ = noise distribution from which negative words are sampled.
- $\mathbb{E}$ = expectation (average over sampled negative words).

The negative sign inside the sigmoid,

$$\sigma(-u_j^T v_c),$$

encourages the dot product between the center word and negative words to be small (or negative). Thus, unrelated words are pushed apart in the embedding space.

Intuition

Suppose the center word is **"king"** and the true context word is **"queen"**.

- Positive pair: ("king", "queen")

$$\log \sigma(u_{\text{queen}}^T v_{\text{king}})$$

- Negative samples: ("king", "table"), ("king", "banana"), ("king", "car")

$$\log \sigma(-u_{\text{table}}^T v_{\text{king}})$$

$$\log \sigma(-u_{\text{banana}}^T v_{\text{king}})$$

$$\log \sigma(-u_{\text{car}}^T v_{\text{king}})$$

The model learns embeddings by **pulling genuine context words closer together while pushing randomly sampled words farther apart**.
For the entire corpus, the objective

$$J(\theta) = \frac{1}{T}\sum_{t=1}^{T} J_t(\theta)$$

simply averages this learning signal over all T training examples. Maximizing $J(\theta)$ produces word vectors that capture semantic and syntactic relationships such as

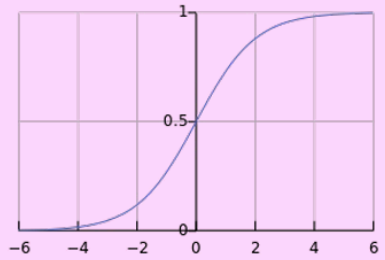
$$\text{king} - \text{man} + \text{woman} \approx \text{queen}.$$

In one sentence: **SGNS learns word embeddings by maximizing the similarity of real center-context pairs and minimizing the similarity of randomly sampled incorrect pairs, using the sigmoid function instead of the computationally expensive softmax.**

The skip-gram model with negative sampling

$$J_{neg-sample}(\boldsymbol{u}_o, \boldsymbol{v}_c, U) = -\log \sigma(\boldsymbol{u}_o^T \boldsymbol{v}_c) - \sum_{k \in \{K \text{ sampled indices}\}} \log \sigma(-\boldsymbol{u}_k^T \boldsymbol{v}_c)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



- We take *k* negative samples (using word probabilities)
- Maximize probability that real outside word appears; minimize probability that random words appear around center word

U(w) is the unigram distribution, meaning how often a word appears in the corpus.
If we sampled negatives directly from U(w)

U(w), we would mostly pick very common words like “the” or “of”.
These frequent words are not informative because they appear in almost every context.
So we reshape the distribution (using $U(w)^{3/4}$) to sample more useful negative words.

3. Why not capture co-occurrence counts directly?

There's something weird about iterating through the whole corpus (perhaps many times); why don't we just accumulate all the statistics of what words appear near each other?!?

Building a co-occurrence matrix *X*

- 2 options: windows vs. full document
- Window: Similar to word2vec, use window around each word → captures some syntactic and semantic information ("word space")
- Word-document co-occurrence matrix will give general topics (all sports terms will have similar entries) leading to "Latent Semantic Analysis" ("document space")

Example: Window based co-occurrence matrix

- Window length 1 (more common: 5–10)
- Symmetric (irrelevant whether left or right context)

- Example corpus:
  - I like deep learning
  - I like NLP
  - I enjoy flying

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

Co-occurrence vectors

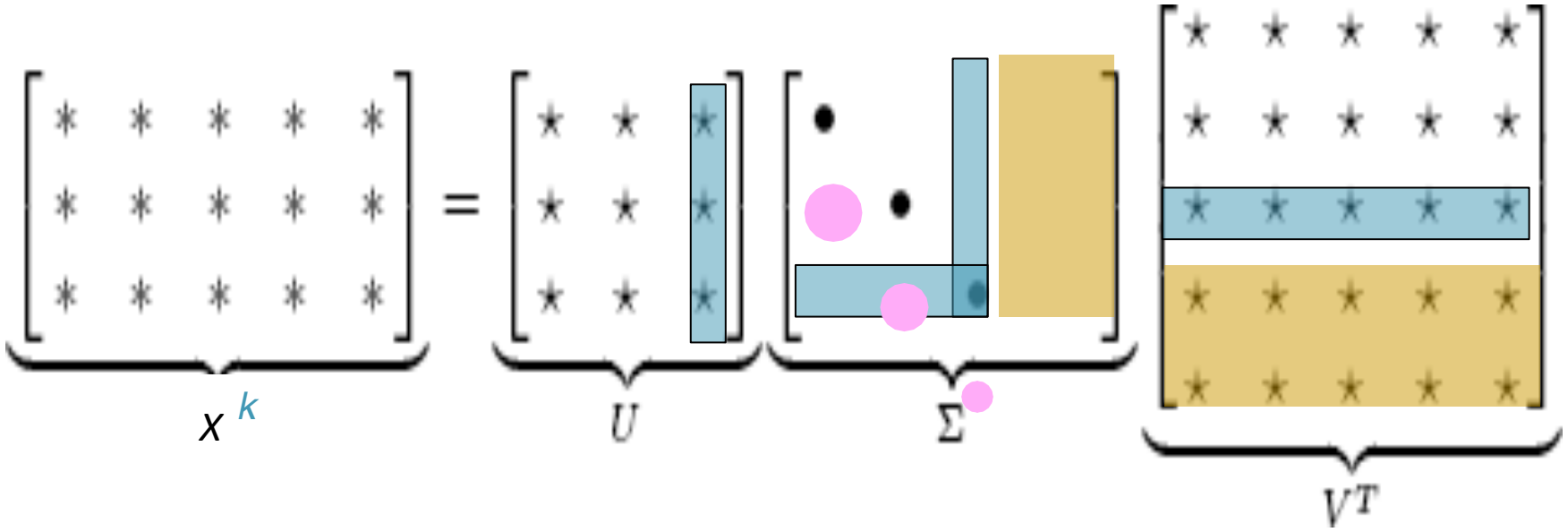
- Simple count co-occurrence vectors
  - Vectors increase in size with vocabulary
  - Very high dimensional: require a lot of storage (though sparse)
  - Subsequent classification models have sparsity issues → Models are less robust

- Low-dimensional vectors
  - Idea: store “most” of the important information in a fixed, small number of dimensions: a dense vector
  - Usually 25–1000 dimensions, similar to word2vec
  - How to reduce the dimensionality?

Classic Method: Dimensionality Reduction on X

Singular Value Decomposition of co-occurrence matrix *X*

Factorizes *X* into $U\Sigma V^T$, where *U* and *V* are orthonormal (unit vectors and orthogonal)



Hacks to X (several used in Rohde et al. 2005 in COALS)

- Running an SVD on raw counts doesn't work well!!!

- Scaling the counts in the cells can help ***a lot***
  - Problem: function words (*the, he, has*) are too frequent → syntax has too much impact. Some fixes:
    - log the frequencies
    - min(*X*, *t*), with $t \approx 100$
    - Ignore the function words
- Ramped windows that count closer words more than further away words
- Use Pearson correlations instead of counts, then set negative values to 0
- Etc.

GloVe (Pennington et al. 2014)

- A matrix factorization approach motivated by ratios of P(word | context) probabilities

Why?

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

- Nice derivation from start to final loss function that satisfies desiderata

Start:

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

Meaningful in linear space (differences, dot products)
Word/context invariance
Robust to low-freq. ctxts.

End:

$$J = \sum_{i,j=1}^{V} f\left(X_{ij}\right)\left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2$$

GloVe (Pennington et al. 2014)

GloVe's idea is that **word meaning is encoded in ratios of co-occurrence probabilities**, not raw counts or local contexts.

So we start with co-occurrence probabilities:

$$P_{ik} = P(w_k|w_i) = \frac{X_{ik}}{X_i}$$

where:

- X_{ik} = how many times word k appears in the context of word i
- $X_i = \sum_k X_{ik}$ = total co-occurrences for word i
- $P(w_k|w_i)$ = probability that word k appears near word i

GloVe (Pennington et al. 2014)

The GloVe paper starts by asking:

> What property of word vectors should capture meaning differences?

They propose:

$$F(w_i, w_j, w_k) = \frac{P_{ik}}{P_{jk}}$$

This ratio expresses how much more likely context word k is to appear with word i than with word j.

GloVe (Pennington et al. 2014)

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

Ratio value	**Meaning**
$\gg$ **1**	(k) is much more associated with **ice**
$\ll$ **1**	(k) is much more associated with **steam**
$\approx$ **1**	(k) does **not** help distinguish them

GloVe (Pennington et al. 2014)

The goal: find a function of word vectors that captures that ratio

We want:

$$F(w_i, w_j, w_k) = \frac{P_{ik}}{P_{jk}}$$

and we assume F depends on the **vector differences** $w_i - w_j$, since meaning relationships are often differences in embedding space (like in Word2Vec: *king – man + woman ≈ queen*).

So the paper proposes:

$$F(w_i, w_j, w_k) = \frac{P_{ik}}{P_{jk}} = \exp\left((w_i - w_j)^T \tilde{w}_k\right)$$

Taking logs:

$$\log P_{ik} = w_i^T \tilde{w}_k + b_i + \tilde{b}_k$$

1. What GloVe tries to learn

GloVe trains word vectors so that:

$$\mathbf{w}_i \cdot \tilde{\mathbf{w}}_j \approx \log(X_{ij})$$

Where:

- X_{ij} = how often word *j* appears near word *i*
- $\mathbf{w}_i$ = vector for center word
- $\tilde{\mathbf{w}}_j$ = vector for context word

Since GloVe learns:

$$\log(X_{ij}) \approx \mathbf{w}_i \cdot \tilde{\mathbf{w}}_j$$

Then:

$$\log(X_{ij}) - \log(X_{kj})$$

becomes approximately:

$$(\mathbf{w}_i - \mathbf{w}_k) \cdot \tilde{\mathbf{w}}_j$$

GloVe (Pennington et al. 2014)

We assume each word w_i and each context w_k has a vector representation:

$$w_i, \tilde{w}_k \in \mathbb{R}^d$$

We want a function F that depends on these vectors — i.e. we can compute F using their embeddings, not using the co-occurrence counts directly.

So:

$$F(w_i, w_j, w_k) = f(w_i, w_j, \tilde{w}_k)$$

GloVe (Pennington et al. 2014)

The GloVe paper makes **three key assumptions** about F:

1. **Only the difference of word vectors should matter**
   (since meaning relationships are relative):

$$F(w_i, w_j, w_k) = f(w_i - w_j, \tilde{w}_k)$$

   because if "ice" and "steam" are similar except in certain contexts ("solid", "gas"), it's their *difference* that matters.
2. **The relationship between co-occurrence and word vectors should be linear** in the simplest possible way.
3. **The model should be scalar-valued** — a single number measuring the ratio.

So they propose a function of the form:

$$f(w_i - w_j, \tilde{w}_k) = (w_i - w_j)^T \tilde{w}_k$$

since the dot product is a simple, scalar, symmetric way to compare vectors.

GloVe (Pennington et al. 2014)

We know the left-hand side must always be **positive** ($P_{ik}, P_{jk} > 0$),

but the right-hand side — a dot product — can be any real number.

So to ensure positivity, we wrap the dot product in an exponential:

$$F(w_i, w_j, w_k) = \exp((w_i - w_j)^T \tilde{w}_k)$$

That's the simplest functional form that satisfies all the above properties.

Summary of steps to implement Glove embeddings

1. **Collect a corpus.**
   Take lots of text (sentences, documents) you want to learn word meanings from.
2. **Decide a context window.**
   For each word token, decide which neighboring words count as its "context" (e.g. ±5 words).
3. **Count co-occurrences.**
   Build a co-occurrence matrix `X`.
   - `X[i, j]` = how many times word `j` appears in the context of word `i`.
     This is a big table of raw counts.
4. **Ask: what do counts tell us?**
   If two words co-occur a lot, they are related. But absolute counts are noisy and biased by word frequency. GloVe uses the log of counts to stabilize things.
5. **Define the model's prediction.**
   Give each word two vectors: a **word vector** `w_i` and a **context vector** `w_j`, plus biases `b_i` and `b_j`.
   The model predicts:

```
prediction_ij = w_i · ~w_j + b_i + ~b_j
```

GloVe (Pennington et al. 2014)

6. **Make the prediction match log-counts.**
We want `prediction_ij` to be close to `log(X[i,j])`. Intuition: dot product should reflect how often the pair co-occurs, but on a log scale.
7. **Weight the importance of pairs.**
Very rare pairs are noisy; very frequent pairs shouldn't dominate. GloVe uses a weighting function `f(X[i,j])` that:

- is ~0 when `X` is 0 (ignore non-occurrences),
- grows for small counts,
- saturates for very large counts.

8. **Minimize squared error (the loss).**
The loss over all word–context pairs is:

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 .$$

Optimize `w`, `w~`, `b`, `b~` with gradient descent.
9. **Extract final word vectors.**
After training you can use `w_i`, or `(w_i + ~ w_i)/2` as the final embedding for word `i`. These vectors encode meaning: similar words end up close in vector space and linear relationships often correspond to semantic relations.

Algorithm: GloVe Word Embedding

Input:
Corpus D
Context window size c
Embedding dimension d
Learning rate α
Number of epochs T

Output:
Word embeddings W

1. PREPROCESS CORPUS
Tokenize the corpus
Build vocabulary V
Assign an index to each word

2. BUILD CO-OCCURRENCE MATRIX
Initialize X[i, j] = 0 for all words i, j

For each sentence in corpus:
For each word wi at position t:
For each context word wj within window c:
distance = |t - position(wj)|
X[i, j] += 1 / distance

3. INITIALIZE PARAMETERS
For each word i in V:
Initialize target vector w[i] randomly
Initialize context vector w_tilde[i] randomly
Initialize biases b[i] = 0
Initialize context biases b_tilde[i] = 0

4. TRAIN MODEL
For epoch = 1 to T:
For each observed pair (i, j) with X[i, j] > 0:

target = log(X[i, j])

prediction =
dot(w[i], w_tilde[j])
+ b[i] + b_tilde[j]

weight = f(X[i, j])

loss =
weight * (prediction - target)^2

Update w[i], w_tilde[j],
b[i], b_tilde[j]
using gradient descent
to minimize loss

5. CREATE FINAL EMBEDDINGS
For each word i in V:
W[i] = w[i] + w_tilde[i]
(Alternatively, use w[i] alone)

Return W

4. How to evaluate word vectors?

- Related to general evaluation in NLP: Intrinsic vs. extrinsic
- Intrinsic:
  - Evaluation on a specific/intermediate subtask
  - Fast to compute
  - Helps to understand that system
  - Not clear if really helpful unless correlation to real task is established
- Extrinsic:
  - Evaluation on a real task
  - Can take a long time to compute accuracy
  - Unclear if the subsystem is the problem or its interaction or other subsystems
  - If replacing exactly one subsystem with another improves accuracy → Winning!

Intrinsic Evaluation of Embeddings (categorization from Schnabel et al 2015)

Intrinsic Evaluation of Word Embeddings
(Schnabel et al., 2015)
Intrinsic evaluations test the *quality* of embeddings **independently** of downstream tasks - i.e., *how well the embeddings capture linguistic or semantic properties*.

1. **Relatedness (Similarity / Relatedness Tasks)**
   - **Goal:** Measure how well embeddings capture semantic similarity between words.
   - **Method:**
     - Compute **cosine similarity** between pairs of word embeddings.
     - Compare it to **human-rated similarity scores** (from datasets like WordSim-353, SimLex-999).
   - **Metric:** Spearman or Pearson correlation.
   - **Example:**
     - Human: *car–automobile* = 0.95 similarity
     - Embedding cosine similarity ≈ 0.92 → **high correlation = good model**

Intrinsic Evaluation of Embeddings (categorization from Schnabel et al 2015)

2. Analogy

- **Goal:** Test whether embedding space encodes **semantic relations** linearly.
- **Famous task:** "a is to b as c is to ?"
- **Method:**
  - Solve by vector arithmetic:

    $\text{vec}(b) - \text{vec}(a) + \text{vec}(c) \approx \text{vec}(d)$
  - Choose word *d* whose embedding is **closest** (by cosine similarity).
- **Example:**
  - "man : king :: woman : ?" → "queen"
- **Datasets:** Google Analogy Test Set, BATS.

Intrinsic Evaluation of Embeddings (categorization from Schnabel et al 2015)

3. Categorization

- **Goal:** See if embeddings group **words of the same category** together.
- **Method:**
  - Cluster embeddings (e.g., with k-means).
  - Measure **cluster purity** or **F-score** compared to gold-standard categories.
- **Example:**
  - Cluster {dog, cat, horse} → animal cluster
  - Cluster {apple, orange, banana} → fruit cluster

Intrinsic Evaluation of Embeddings (categorization from Schnabel et al 2015)

4. Selectional Preference

- **Goal:** Measure how well embeddings capture **typical syntactic-semantic relations** (like verb–object pairs).
- **Method:**
  - Compute compatibility between **verb** and **noun** embeddings.
  - Check whether typical pairs (e.g., *eat-apple*) score higher than atypical ones (e.g., *eat-rock*).
- **Interpretation:**
  - Tests whether embeddings encode plausible argument structure and semantics.

Extrinsic Evaluation: Using Word Embeddings in Systems

Embedding Initialization and Extrinsic Evaluation

1. Initializing with Pre-trained Embeddings

- **Idea:** Start your model with **pre-trained word embeddings** (e.g., Word2Vec, GloVe, FastText) instead of random initialization.
- **Why:**
  - Pre-trained embeddings already capture **semantic and syntactic information**.
  - This leads to **faster convergence** and **better generalization**, especially with limited data.
- **Options:**
  - **Fixed embeddings:** Use pre-trained vectors and **don't update** them during training.
    - Pros: Fewer parameters, less risk of overfitting.
    - Cons: May not adapt to task-specific meaning.
  - **Fine-tuned embeddings:** Start with pre-trained, but **allow updates** during training.
    - Pros: Task-adaptive.
    - Cons: Risk of "forgetting" general knowledge.

Extrinsic Evaluation: Using Word Embeddings in Systems

2. Concatenating Pre-trained and Learned Embeddings

- Combine **two representations per word:**
  - One from **pre-trained embeddings** (captures general meaning).
  - One from **learned embeddings** (captures task-specific nuances).
- **Advantages:**
  - More expressive — merges general + task-specific info.
- **Disadvantages:**
  - Increases **model size** and **computation cost.**

Example:

For each word → `[pretrained_vector ; learned_vector]`

Final embedding dimension doubles → more expressive, but heavier.

Extrinsic Evaluation: Using Word Embeddings in Systems

3. Extrinsic Evaluation: Using Embeddings in Systems

- **Definition:**
  Evaluate how useful embeddings are when used as **input features** for downstream NLP tasks.
  (Contrast with *intrinsic*, which evaluates embeddings alone.)
- **Examples of Extrinsic Tasks:**
  - **Sentiment analysis** (Do embeddings help classify reviews?)
  - **Named Entity Recognition (NER)**
  - **Machine translation**
  - **Question answering**
- **Goal:**
  Measure **end-task performance** (e.g., accuracy, F1, BLEU) to assess whether embeddings actually **improve real-world results.**

How Do I Choose Embeddings?

- No one-size-fits-all embedding (Schnabel et al 2015)

	relatedness						categorization			sel. prefs		analogy			
	rg	ws	wss	wsr	men	toefl	ap	esslli	batt.	up	mcrae	an	ansyn	ansem	average
CBOW	**74.0**	**64.0**	**71.5**	**56.5**	**70.7**	66.7	**65.9**	**70.5**	**85.2**	24.1	13.9	**52.2**	**47.8**	**57.6**	**58.6**
GloVe	63.7	54.8	65.8	49.6	64.6	**69.4**	64.1	65.9	77.8	27.0	**18.4**	42.2	44.2	39.7	53.4
TSCCA	57.8	54.4	64.7	43.3	56.7	58.3	57.5	**70.5**	64.2	**31.0**	14.				
C&W	48.1	49.8	60.7	40.1	57.5	66.7	60.6	61.4	80.2	28.3	16.				
H-PCA	19.8	32.9	43.6	15.1	21.3	54.2	34.1	50.0	42.0	-2.5	3.				
Rand. Proj.	17.1	19.5	24.9	16.1	11.3	51.4	21.9	38.6	29.6	-8.5	1.				

Table 1: Results on absolute intrinsic evaluation. The best result for each
The second row contains the names of the corresponding datasets.

	dev	test	p-value
Baseline	94.18	93.78	0.000
Rand. Proj.	94.33	93.90	0.006
GloVe	94.28	93.93	0.015
H-PCA	94.48	93.96	0.029
C&W	**94.53**	**94.12**	
CBOW	94.32	93.93	0.012
TSCCA	**94.53**	94.09	0.357

Table 4: F1 chunking results using different word embeddings as features. The p-values are with respect to the best performing method.

- Be aware, and use the best one for the task

Improving Embeddings

1. Sensitive to superficial differences (dog / dogs)

Word embeddings like Word2Vec or GloVe treat each distinct word form as a separate entity.
This means "dog" and "dogs" are represented by different vectors, even though they have nearly the same meaning.
They do not understand that the "-s" suffix marks plural form.

Example:

- The embeddings for "dog" and "dogs" are closer to each other than to "cat,"
  but they are still treated as separate points in space.

Why it happens:
Because these models operate at the word level, they ignore subword or morphological structure.

Solution:
Models like FastText improve on this by using character n-grams, allowing shared representations between related words like "dog" and "dogs."

Improving Embeddings

2. Not necessarily coordinated with knowledge or across languages

Embeddings capture statistical patterns in text, not factual or logical knowledge.
So they may represent words as similar because they occur in similar contexts, even if that's conceptually wrong.

Example:
"whale" may be close to "fish" in embedding space because both appear in contexts like "ocean," not because whales are fish.
Similarly, embeddings trained in different languages (English, Spanish, etc.) are not aligned unless explicitly trained to be so - "dog" and "perro" might end up far apart.

Improving Embeddings

3. Not interpretable

Embeddings are high-dimensional vectors (e.g., 300 numbers).
There's no clear meaning attached to any individual dimension.
You can measure relationships (like distance or direction), but you can't explain what a particular dimension represents.

Example:
You can find patterns such as "king – man + woman ≈ queen," but you can't say what axis corresponds to "gender" or "royalty."

Improving Embeddings

4. Can encode bias

Embeddings learn from real-world text data, which contains social and cultural biases.
As a result, these models often reproduce or amplify stereotypes.

Example:
In early Word2Vec models trained on Google News:
“man : computer programmer :: woman : homemaker.”

This happens because of the way words co-occur in biased text.

Possible approaches:

- Post-processing or debiasing techniques
- Using balanced or filtered corpora

Sub-word Embeddings (1)

Example: "cats"
If we use **character-based embeddings**, the word "cats" becomes:

`c a t s`
Each character gets a small vector from a **character lookup table**.
These vectors are passed through a **bidirectional LSTM (BiLSTM)** or CNN to compose them into a single word-level vector.
So instead of a fixed lookup, the model *learns* how to combine subword information to produce a word embedding.
This means:

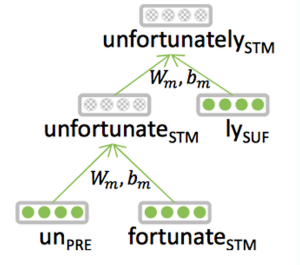
- "cat" and "cats" will end up close in vector space, because they share most subword patterns.
- Even unseen words (e.g., "catsy") can still get embeddings, based on their character composition.

Morpheme-based version
Similarly, a **morpheme-based model** splits the word into meaningful units (prefixes, roots, suffixes):
Example:
`unfortunately → un + fortunate + ly`
Each morpheme has an embedding.
These are combined (often using an RNN) to produce the word's vector.
So:

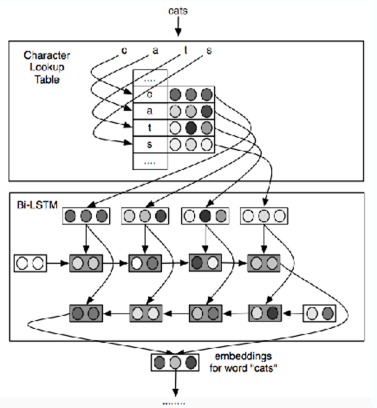
- "unfortunate" and "fortunately" share the root "fortunate."
- This allows the model to capture *morphological regularities*.

- Can capture sub-word regularities

Morpheme-based (Luong et al. 2013)



Character-based (Ling et al. 2015)



Subword-level idea

Instead of looking up a single word embedding, we:

1. Break the word into smaller pieces (characters or morphemes).
2. Generate embeddings for those pieces.
3. Combine them (e.g., using an LSTM, CNN, or average) to form the full word's embedding.

Sub-word Embeddings (2)

Regular word embeddings (like Word2Vec, GloVe) treat each word as a single token.
→ This causes problems with:

- Rare words
- Morphological variants (e.g., "walk," "walking")
- Unseen words (OOV = out of vocabulary)

To solve this, **we can represent each word using its character n-grams** — sequences of *n* consecutive characters.

The core idea
A word is represented **as the sum (or average)** of the embeddings of its **character n-grams**.
This captures subword structure such as prefixes, suffixes, and roots.

Example
Take the word:
`where`
We add boundary markers < and > to indicate the start and end of the word:

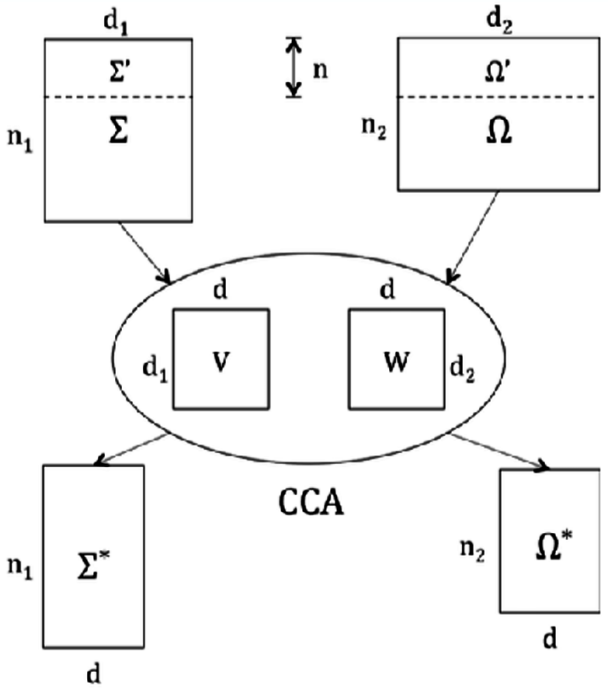
`<where>`
Now we extract all **character n-grams** of length **3 to 6**:
And also include the **whole word** itself ("where").

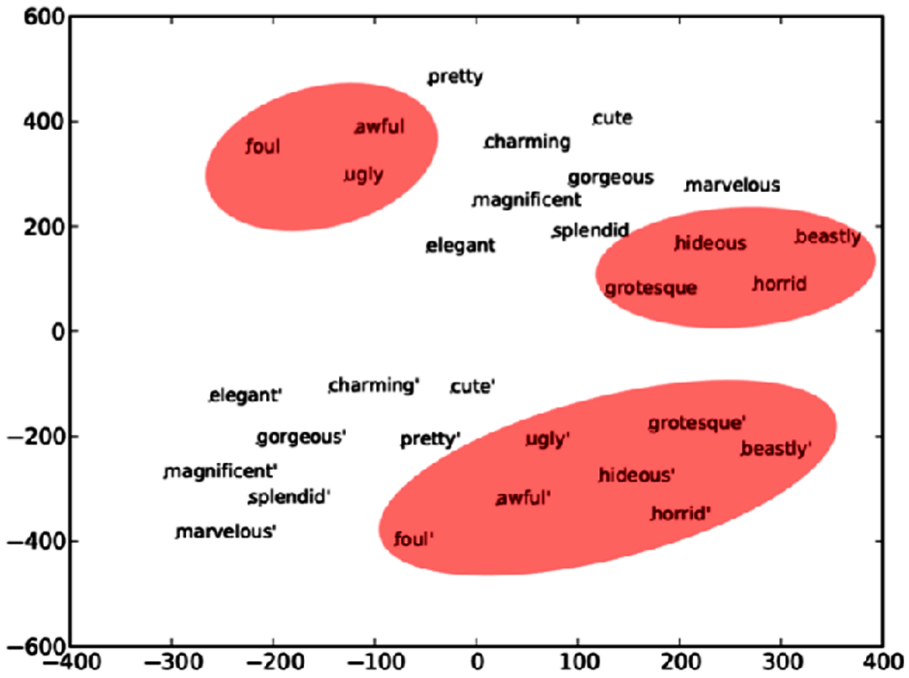
Representation
Each n-gram has its own learned embedding vector.
The **word's embedding** is obtained by combining (typically summing or averaging) all its n-gram embeddings:

n	n-grams
3	<wh, whe, her, ere, re>
4	<whe, wher, here, ere>
5	<wher, where, here>
6	<where, where>

Multilingual Coordination of Embeddings (Faruqui et al. 2014)

- We have word embeddings in two languages, and want them to match





Unsupervised Coordination of Embeddings

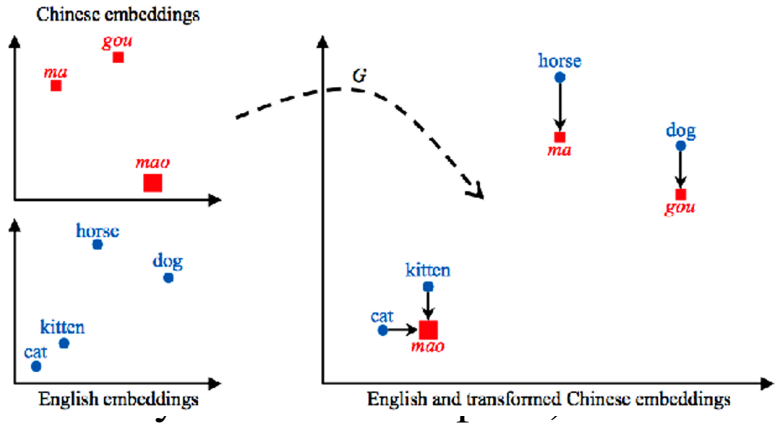
The Goal - Coordinate Embeddings Across Languages

Problem:

- Word embeddings trained separately on different languages are *not aligned*.
- So "dog" (English) and "chien" (French) may be far apart in vector space.

Goal:

- Map embeddings from one language to another **without parallel data** (i.e., r



The Surprising Idea - No Dictionary Needed

Traditional way: Use a bilingual dictionary to align words.

Unsupervised way:

- Find a transformation that aligns the *geometry* (structure) of two embedding spaces.
- Assume both spaces have **similar shapes** because word co-occurrence patterns are universal.

Reference:

- *Artetxe et al., 2017*: "Learning bilingual mappings with identical words."
- *Zhang et al., 2017*: "Adversarial training for unsupervised cross-lingual mapping."

Unsupervised Coordination of Embeddings

Artetxe et al. (2017) - Identical Words Trick

Key insight:

Even without a dictionary, there are some **identical words** (shared strings) -
like numbers, names, punctuation, or brand names (e.g., “COVID,” “2025,” “UN,” “Google”).

Steps:

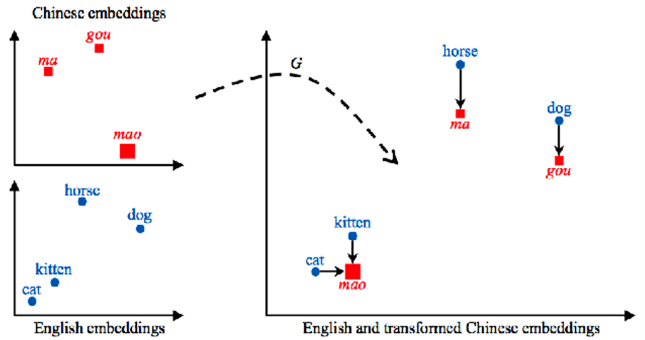
1. Use identical words (digits, names, etc.) as anchor points.
2. Learn a **linear mapping** W to align source embeddings X and target embeddings Y

$$W^* = \arg\min_W \|WX - Y\|$$

3. Refine iteratively using mutual nearest neighbors.

Outcome:

Aligned bilingual embedding spaces without explicit supervision.



Unsupervised Coordination of Embeddings

Zhang et al. (2017) - Matching Distributions

Idea:
Instead of aligning individual words, align **entire distributions** of embeddings.

Method:

- Use **adversarial training** (GAN-like setup):
  - A *generator* learns the mapping W between languages.
  - A *discriminator* tries to tell whether a word embedding is from the source or target language.
  - Training continues until the discriminator can't tell them apart.

Effect:
The two spaces become statistically indistinguishable - achieving cross-lingual alignment *unsupervised*.

Sparse Embeddings

The Problem

- Standard word embeddings (like Word2Vec, GloVe, BERT) are **dense** vectors - every dimension has some small value.
- These dimensions are **not interpretable** — you can't say, "dimension 1 = gender," "dimension 2 = size," etc.
- Each vector encodes information in a distributed way, making analysis difficult.

Model	Top 5 Words (per dimension)
SVD_{300}	well, long, if, year, watch plan, engine, e, rock, very get, no, features, music, via features, by, links, free, down works, sound, video, building, section
$NNSE_{1000}$	inhibitor, inhibitors, antagonists, receptors, inhibition bristol, thames, southampton, brighton, poole delhi, india, bombay, chennai, madras pundits, forecasters, proponents, commentators, observers nosy, averse, leery, unsympathetic, snotty

2. The Idea - Sparsity for Interpretability

- To make embeddings more interpretable, researchers proposed **sparse embeddings**.
- In a **sparse vector**, most values are **zero**, and only a few dimensions have meaningful (non-zero) values.
- Each active dimension tends to represent a more distinct, interpretable feature (e.g., "is animal," "is profession," "is location").

The Method

- Add a **sparsity constraint** during training.
- This can be done by adding a **regularization term** to the loss function (e.g., L1 regularization) that encourages many weights to be zero.

Mathematically:

$$\text{Loss} = \text{original loss} + \lambda \sum_i |w_i|$$

where λ controls the degree of sparsity.

De-biasing Word Embeddings (Bolukbasi et al. 2016)

1. The Problem: Bias in Word Embeddings

- Word embeddings (Word2Vec, GloVe, etc.) are learned from real-world text.
- Text data reflects **societal biases** (gender, race, occupation, etc.).
- As a result, embeddings capture and reproduce these biases.

Example:

man : computer programmer :: woman : homemaker

This shows that the model has learned gender stereotypes from data.

Extreme *she*	Extreme *he*
1. homemaker	1. maestro
2. nurse	2. skipper
3. receptionist	3. protege
4. librarian	4. philosopher
5. socialite	5. captain
6. hairdresser	6. architect
7. nanny	7. financier
8. bookkeeper	8. warrior
9. stylist	9. broadcaster
10. housekeeper	10. magician

Gender stereotype *she-he* analogies		
sewing-carpentry	registered nurse-physician	housewife-shopkeeper
nurse-surgeon	interior designer-architect	softball-baseball
blond-burly	feminism-conservatism	cosmetics-pharmaceuticals
giggle-chuckle	vocalist-guitarist	petite-lanky
sassy-snappy	diva-superstar	charming-affable
volleyball-football	cupcakes-pizzas	lovely-brilliant
Gender appropriate *she-he* analogies		
queen-king	sister-brother	mother-father
waitress-waiter	ovarian cancer-prostate cancer	convent-monastery

2. The Core Idea: Debiasing

We can try to **remove or reduce bias** by identifying and neutralizing the bia

Solution:

1. Identify bias-defining word pairs (e.g., gender pairs).
2. Compute the bias direction.
3. Neutralize words that should be unbiased.

Result:

Embeddings become more **balanced**, though perfect debiasing remains challenging.

A Case Study: FastText

What is FastText?

FastText (by Facebook AI Research, 2016) is an **improved version of Word2Vec** that fixes a big weakness in Word2Vec:
Word2Vec can't handle new or rare words well.

The Problem with Word2Vec

Word2Vec treats each word as an **atomic symbol** — like "dog", "dogs", "doggy" are all **different words**.

So:

- It learns **separate embeddings** for "run", "running", "runner", etc.
- If a new word appears (say, "runfast"), it has **no vector** — it's *OOV* (out-of-vocabulary).

FastText's Big Idea

Instead of learning vectors **only for whole words**,
FastText also learns from **subword information** (character n-grams).

A word = sum (or average) of its **character n-gram vectors**

A Case Study: FastText

Example:
Let's take the word **"playing"**
and n-grams of size 3 (trigrams).
FastText breaks it down as:

`<pl, pla, lay, ayi, yin, ing, ing>`
(Usually with boundary symbols "<" and ">" to mark word start/end.)

Then:
Each trigram (like "pla", "lay", "ing") has its **own vector**.
The word vector for **"playing"** is:

$$v_{\text{playing}} = \sum_{g \in \text{ngrams(playing)}} z_g$$

vplaying =g∈ngrams(playing)∑ zg
Where:

- z_g = vector for character n-gram g
- The model sums or averages all n-gram vectors to get the final word vector.

A Case Study: FastText

How Training Works

It still uses **the Skip-Gram (or CBOW)** model - same as Word2Vec!

But instead of updating the vector for the word “playing”,
it updates **all the subword vectors** that compose it.

So training pairs look like:

```
Input: n-grams of "playing"
Output: context words (like “football”, “is”, “he”)
```